

**WYDZIAŁ TELEKOMUNIKACJI, INFORMATYKI
I ELEKTROTECHNIKI**

Programowanie aplikacji mobilnych [05-IST-PAM-SP5] - [laboratorium](#)

SPRAWOZDANIE Z LABORATORIUM

Lab (nr_10) Temat sprawozdania: **Aplikacja Kotlin cz 2**



Wykonali:

Paweł Jońca **05-IST-PAM-SP5** pawjon000@pbs.edu.pl

(05-IST-PAM-SP5) 2025_ rok 3 sem. 5 gr 3

Data: 2025-12-11 (Data laboratorium)

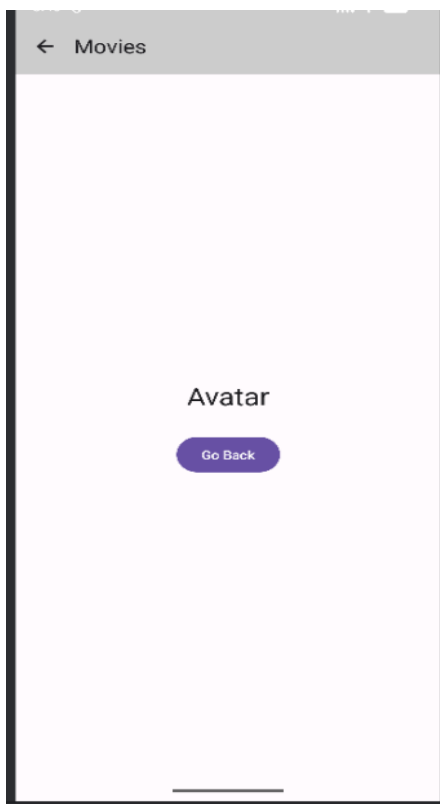
Cel zadania

Celem zadania było rozwinięcie wcześniej przygotowanej aplikacji mobilnej napisanej w Kotlinie tak, aby potrafiła wyświetlać na ekranie telefonu listę elementów tekstowych oraz umożliwiała przechodzenie do osobnego widoku prezentującego szczegóły wybranego elementu. Oprócz samego pokazania listy ważne było poprawne zorganizowanie struktury programu zgodnie z architekturą Model–Widok–Kontroler (MVC). Oznacza to, że warstwa odpowiedzialna za wygląd ekranu miała być oddzielona od logiki sterującej przechodzeniem między ekranami i przekazywaniem danych.

Na ekranie głównym użytkownik widzi zestaw przykładowych nazw, przedstawionych w formie przewijanej listy. Każda pozycja reaguje na kliknięcie. Po wybraniu elementu aplikacja przełącza się na drugi ekran, na którym wyświetlana jest nazwa klikniętej pozycji jako jej „szczegóły”. Dane te są przekazywane pomiędzy ekranami przez mechanizm nawigacji. Widok szczegółów nie pobiera sam danych ani nie podejmuje decyzji o przejściu między ekranami – jedynie odbiera tekst i go wyświetla, dzięki czemu spełnia rolę czystej warstwy prezentacji.

Realizacja zadania wymagała dodania nowego ekranu szczegółów oraz rozszerzenia konfiguracji nawigacji o nową trasę z parametrem tekstowym. Dodatkowo należało zapewnić możliwość powrotu do ekranu głównego zarówno przyciskiem w górnym pasku, jak i przyciskiem umieszczonym w treści widoku.

Zadanie 1



Wnioski

Podczas pracy okazało się, że dodanie nowego widoku wymaga zmian nie tylko w samym pliku tego widoku, ale także w centralnym module odpowiedzialnym za nawigację. To właśnie tam definiuje się, jakie ekrany istnieją i jakie dane można do nich przekazywać. Ekran główny wywołuje nawigację i przekazuje wybrany tekst, natomiast ekran szczegółów jedynie go odbiera i prezentuje.

Takie rozdzielenie ról potwierdza zastosowanie architektury MVC: lista danych pełni funkcję modelu, pliki ekranów są widokami, a komponent nawigacji działa jak kontroler zarządzający przepływem aplikacji.

Dzięki temu kod jest bardziej czytelny i łatwiejszy do rozbudowy, ponieważ zmiany w wyglądzie ekranu nie wpływają na logikę przechodzenia między widokami.

Efektem ćwiczenia jest prosta aplikacja wieloekranowa z działającą nawigacją i przekazywaniem danych między ekranami, zbudowana w sposób uporządkowany i zgodny z dobrymi praktykami projektowania aplikacji mobilnych.

DetailsScreen.kt

```
package com.example.lab9kotlin.details
```

```
import androidx.compose.foundation.layout.*
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ArrowBack
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.unit.dp
import androidx.navigation.NavController
```

```
@OptIn(ExperimentalMaterial3Api::class)
```

```
@Composable
```

```
fun DetailsScreen(
```

```
    navController: NavController,
```

```
    movieData: String?
```

```
) {
```

```

Scaffold(
  topBar = {
    TopAppBar(
      title = { Text(text = "Movies") },
      navigationIcon = {
        IconButton(onClick = { navController.popBackStack() }) {
          Icon(
            imageVector = Icons.Default.ArrowBack,
            contentDescription = "Back"
          )
        }
      },
      colors = TopAppBarDefaults.topAppBarColors(
        containerColor = Color.LightGray
      )
    )
  }
) { paddingValues ->

```

```

Column(
  modifier = Modifier
    .fillMaxSize()
    .padding(paddingValues),
  horizontalAlignment = Alignment.CenterHorizontally,
  verticalArrangement = Arrangement.Center
) {

```

```

Text(
  text = movieData ?: "No Data",
  style = MaterialTheme.typography.headlineMedium
)

```

```

        Spacer(modifier = Modifier.height(24.dp))

        Button(onClick = { navController.popBackStack() }) {
            Text(text = "Go Back")
        }
    }
}
}

```

MovieNavigation.kt

```

package com.example.lab9kotlin.navigation

import androidx.compose.runtime.Composable
import androidx.navigation.NavType
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import androidx.navigation.navArgument
import com.example.lab9kotlin.details.DetailsScreen
import com.example.lab9kotlin.screens.home.HomeScreen

@Composable
fun MovieNavigation() {
    val navController = rememberNavController()

    NavHost(
        navController = navController,
        startDestination = MovieScreens.HomeScreen.name
    ) {

```

```

composable(route = MovieScreens.HomeScreen.name) {
    HomeScreen(navController = navController)
}

composable(
    route = MovieScreens.DetailsScreen.name + "{movie}",
    arguments = listOf(
        navArgument("movie") { type = NavType.StringType }
    )
) { backStackEntry ->

    val movieData = backStackEntry.arguments?.getString("movie")

    DetailsScreen(
        navController = navController,
        movieData = movieData
    )
}
}

```

HomeScreen.kt

```

package com.example.lab9kotlin.screens.home

import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier

```

```

import androidx.compose.ui.graphics.Color
import androidx.compose.ui.unit.dp
import androidx.navigation.NavController
import com.example.lab9kotlin.components.MovieRow

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun HomeScreen(navController: NavController) {

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Movies TopAppBar Wykład") },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = Color.Magenta,
                    titleContentColor = Color.White
                )
            )
        },
        bottomBar = {
            BottomAppBar(containerColor = Color.Blue) {
                Text(
                    text = "Movies bottomBar Wykład",
                    modifier = Modifier.padding(start = 16.dp),
                    color = Color.White
                )
            }
        }
    ) { innerPadding ->

        Column(modifier = Modifier.padding(innerPadding)) {

```

```

        MainContent(navController = navController)
    }
}

```

@Composable

```

fun MainContent(
    navController: NavController,
    moviesList: List<String> = listOf(
        "Avatar",
        "555",
        "Harry PotterX",
        "Life",
        "Lolek",
        "Bolek",
        "Krecik",
        "Idą Święta",
        "Wykład"
    )
){
    Column(modifier = Modifier.padding(12.dp)) {

        LazyColumn {
            items(items = moviesList) { movie ->

                MovieRow(movie = movie) { selectedMovie ->
                    navController.navigate("DetailsScreen/$selectedMovie")
                }
            }
        }
    }
}

```

}